produces:

```
Called wibble with [] and
Called wobble with [1, 2] and
Called wurble with [3, 4] and #<Proc:0x0000000104df8888 prog.rb:7>
```

Before we get too deep into the details, we'll offer a tip about etiquette. There are two main ways that people use method_missing. The first intercepts every use of an undefined method and handles it. The second is more subtle; it intercepts all calls but handles only some of them. In the latter case, it is important to forward on the call to a superclass if you decide not to handle it in your method_missing implementation.

In this snippet, imagine there is some method called name_handled_by_method_missing? that determines if the name matches whatever criteria we are using for dynamic methods:

```
class MyClass < OtherClass
  def method_missing(name, *args, &block)
    if name_handled_by_method_missing?(name)
      # handle call
    else
      super   # otherwise pass it on
    end
  end

  def respond_to_missing?(name)
    name_handled_by_method_missing?(name)
  end
end
```

If you fail to pass on calls that you don't handle, your application will silently ignore calls to unknown methods in your class.

It's also important to keep the respond_to_missing? method consistent with method_missing. As we mentioned in Duck Typing, on page 357, respond_to? is used to query what messages an instance expects. Using method_missing changes the list of expected messages, so there's a parallel to respond_to? called respond_to_missing? that is similarly invoked if respond_to? returns false, as a final check on whether the class actually responds to the message.

Using method_missing to Simulate Accessors

The OpenStruct class is distributed with Ruby. It allows you to write objects with attributes that are created dynamically by assignment. For example, you could write this:

```
require 'ostruct'
obj = OpenStruct.new(name: "Dave")
obj.address = "Texas"
obj.likes   = "Programming"

puts "#{obj.name} lives in #{obj.address} and likes #{obj.likes}"
```

produces:

```
Dave lives in Texas and likes Programming
```

Let's use method_missing to write our own version of OpenStruct:

metaprogramming/open_struct.rb

```
class MyOpenStruct < BasicObject
  def initialize(initial_values = {})
    @values = initial_values
```